
Improved version:

```
#include <iostream>
using namespace std;

int main() {

    int access[100];
    int cache[100];

    int n;
    int cacheSize;
    int currentSize = 0;

    double hit = 0;
    double miss = 0;

    cout << "Enter cache size: " << endl;
    cin >> cacheSize;

    // Input validation
    if (cacheSize <= 0 || cacheSize >= 100) {
        cout << "Invalid cache size!" << endl;
        return 1;
    }

    cout << "Enter number of memory accesses: " << endl;
    cin >> n;

    // Input validation
    if (n <= 0 || n > 100) {
        cout << "Invalid number of memory accesses!" << endl;
        return 1;
    }

    cout << "Enter memory addresses:" << endl;

    for (int i = 0; i < n; i++) {
        cin >> access[i];
    }

    int i = 0;
```

```

// Main simulation loop
while (i < n) {

    int addr = access[i];

    bool haveFound = false;
    int position = -1;

    // Search cache
    for (int j = 0; j < currentSize; j++) {

        if (cache[j] == addr) {
            haveFound = true;
            position = j;
            break;
        }
    }

    // HIT
    if (haveFound) {

        cout << addr << " -> HIT" << endl;
        hit++;

        // Move item to back (Most Recently Used)

        for (int j = position; j < currentSize - 1; j++) {
            cache[j] = cache[j + 1];
        }

        cache[currentSize - 1] = addr;
    }

    // MISS
    else {

        cout << addr << " -> MISS" << endl;
        miss++;

        // If cache still has space
        if (currentSize < cacheSize) {

            cache[currentSize] = addr;

```

```

        currentSize++;
    }

    // Remove Least Recently Used item
    else {

        for (int j = 0; j < cacheSize - 1; j++) {
            cache[j] = cache[j + 1];
        }

        cache[cacheSize - 1] = addr;
    }
}

// Print components of cache
cout << "Cache: ";

for (int j = 0; j < currentSize; j++) {
    cout << cache[j] << " ";
}

cout << "\n" << endl;

i++;
}

cout << "Total hits: " << hit << endl;
cout << "Total misses: " << miss << endl;

double hitRate = (hit / (hit + miss)) * 100;

cout << "Hit rate: " << hitRate << "%" << endl;

return 0;
}

```